Data structures: contracts, representations, and cost

A data structure is more than a storage shape. Each operation needs a contract, a state transition, and a cost model. This chapter develops all three using traceable examples.

Algorithms and programs

An algorithm gives unambiguous steps and terminates on legal inputs. A service program may run indefinitely. Here termination is part of each operation’ s contract; empty input is legal only when its behavior is specified.

Abstract values before storage

An abstract data type (ADT) specifies observable values and operations; a representation describes storage. Implementations may arrange memory differently while satisfying the same observable contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A bounded stack contract

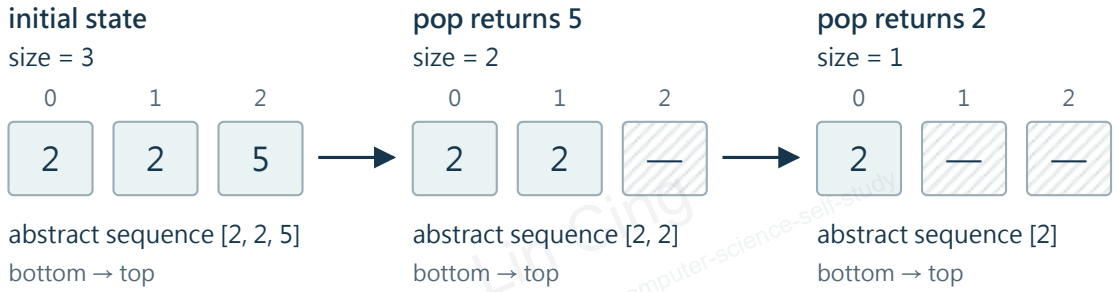
For fixed capacity $C \geq 0$, the abstract value is a finite sequence S of length at most C, with the top at the right. create(C) returns empty; isEmpty and isFull test lengths 0 and C. A successful push(x) returns success and produces $S \parallel [x]$; a successful pop returns and removes the last element. Push when full returns failure; pop when empty reports failure. Both leave S unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicates matter

A stack is an ordered sequence, not a membership set: one pop from $[2, 2]$ leaves $[2]$. The figure connects array slots to the ordered abstract sequence.

Capacity 3: an array representation maps to a sequence



α(a, size) = [a[0], ..., a[size − 1]]

Hatched slots are outside the abstract value; equal elements occupy distinct positions.

Representation invariant and abstraction function

For array a, capacity C and integer s, a legal state satisfies $0 \le s \le C$, and $a[0], \ldots, a[s-1]$ hold valid elements. The abstraction function is $\alpha(a, s) = [a[0], \ldots, a[s-1]]$. Stale values in $a[s..C)$ are not stack elements; do not read $a[C]$.

Proving push preserves the contract

Assume the invariant and $s < C$. First successfully write $a[s] = x$, then set $s \leftarrow s + 1$. The length stays within capacity and the old prefix is unchanged, so the new abstract value is the old value followed by x. If full, report failure before modifying the state.

Pop and invalid states

When $s > 0$, obtain $a[s-1]$ then decrement s; the abstract sequence loses precisely its old last element. When $s = 0$, do not read $a[-1]$. A stale old slot is outside the abstract value; $s > C$ is an invalid representation to which the abstraction function does not apply.

C++ values and exceptions

An empty pop can return `std::optional<T>` or report a specified exception. Copying or moving `T` and allocating storage may throw; do not change `size` first and assume success. The unchanged-on-failure guarantee requires suitable non-throwing operations or an explicit rollback strategy.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Different search contracts

Here lower_bound uses a 0-based nondecreasing array and returns the first position $i \in [0, n]$ whose element is not less than x, or n when none exists. The Algorithms course FIRST-MATCH uses 1-based indices and returns NIL when absent. An insertion position of n is not the same contract as NIL.

The complete lower_bound invariant

For natural nondecreasing order, maintain $0 \le l \le r \le n$; every $0 \le j < l$ satisfies $A[j] < x$, and every $r \le j < n$ satisfies $A[j] \ge x$. Unclassified elements are in $[l, r)$; candidate insertion positions lie in the closed interval $[l, r]$. Return the first not-less position even with duplicates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

lower_bound: an executable update rule

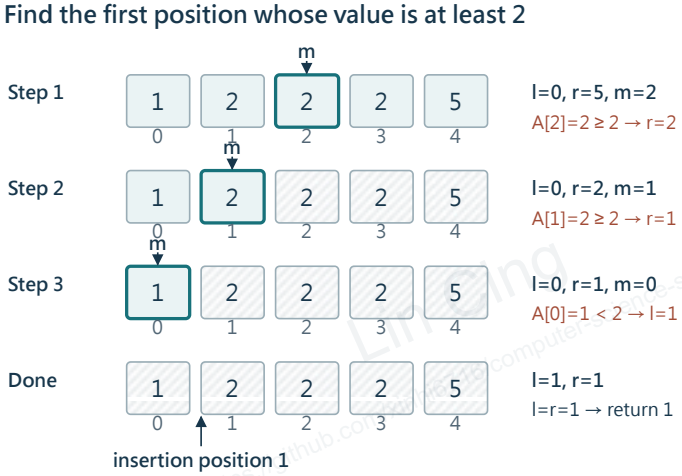
Input $A[0..n)$ is sorted in nondecreasing order. Return the first position whose value is not less than x. The right endpoint is exclusive and starts at n.

```
l ← 0;  r ← n
while l < r:
    m ← l + floor((r - l) / 2)
    if A[m] < x:  l ← m + 1
    else:         r ← m
return l
```

Initially both classified regions are empty. Each update classifies one side while retaining possible insertion positions. If every element is less than x, return n without reading $A[n]$.

Three search steps

For $A = [1, 2, 2, 2, 5]$ and $x = 2$, every update preserves the invariant. Stop at $l = r = 1$. An empty array returns 0; $x = 6$ returns 5 without reading $A[5]$.



Solid: unclassified elements [l,r); candidate positions [l,r].
Hatching marks elements, not candidate positions.

Search invariant: initialization and both updates

Let k be the first position whose value is not less than x, or n if none exists. We prove $k \in [l, r]$ at the start of every iteration.

- **Initialization:** $l = 0, r = n$, so $k \in [0, n]$. Both classified regions are empty.
- **If $A[m] < x$:** sortedness gives $A[0..m] < x$, so $k \geq m + 1$. Setting $l = m + 1$ retains k.
- **If $A[m] \geq x$:** sortedness gives $A[m..n) \geq x$, so $k \leq m$. Setting $r = m$ still retains k.
- **Termination:** $l = r$ leaves the single candidate $k = l$. The proof also covers $k = n$ and duplicate keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why search terminates

When $l < r$, $m = l + \lfloor (r - l)/2 \rfloor$ satisfies $l \leq m < r \leq n$, so $A[m]$ is valid. If $A[m] < x$, classify elements $[l, m]$ on the left; otherwise classify elements $[m, r)$ on the right. Position m can still be the answer. The nonnegative integer $r - l$ decreases each iteration; at $l = r$, the classified intervals imply the postcondition.

Comparators and executable semantics

A C++ comparator comp must be a strict weak ordering, and the array must be sorted by it. The left region satisfies comp(A[j],x), the right region !comp(A[j],x); the result is the first position where comp(A[i],x) is false. Check $i < n$ before reading A[i]; equivalence then requires both !comp(A[i],x) and !comp(x,A[i]), not necessarily ==. The index type must represent n and the midpoint arithmetic without overflow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparisons are not the whole cost

With random access and $O(1)$ comparisons, lower_bound takes $\Theta(\log(n+1))$ worst-case comparisons. If strings have at most L characters, each lexicographic comparison uses $O(L)$ character operations, giving $O(L\log(n+1))$ character operations. Including indexing and control costs, total time is $O((L+1)(1+\log(n+1)))$. An empty array makes no comparisons but still incurs constant control cost.

State the cost model

Separate fixed storage, input, output, and auxiliary space. Time may count comparisons, element moves, or bit operations. C++ `vector` allocation and construction costs depend on the element type and allocator; a unit-cost model must say so.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Additions versus bit complexity

Add n integers to an accumulator in sequence. If **both operands and the result of every addition** fit in at most L bits, grade-school bitwise addition costs $O(L)$ per step and $O(nL)$ total; $n=0$ has only initialization cost.

A short final result is not enough. For example, $2^N+(-2^N)=0$ has a constant-length result, but both inputs have $\Theta(N)$ bits; reading the complete input can already require $\Omega(N)$ time.

With fixed-width machine words and a guarantee of no overflow, a word-RAM model may instead charge $O(1)$ per addition, giving $O(n)$ total time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recursion and live frames

For nonnegative n, $fac(0) = 1$ and $fac(n) = n\,fac(n-1)$. Evaluating $fac(3)$ calls 3, 2, 1, 0: four calls, three edges, and four simultaneously live frames at the deepest point. Returns are 1, 1, 2, 6. We count each call once and do not treat arbitrary-precision multiplication as free.

Two Fibonacci conventions

With $F(0) = 0, F(1) = 1, F(5) = 5$; with $G(0) = G(1) = 1$ and the same recurrence, $G(5) = 8$. Counting the initial direct-recursion call gives $C(0) = C(1) = 1$ and $C(n) = 1 + C(n-1) + C(n-2)$, so $C(5) = 15$ and $C(8) = 67$. Total calls and peak frame depth are different quantities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplying by two is not two calls

In a source-level unit-cost model, for $n = 2^k \geq 1$ and $T(1) = \Theta(1)$, `2*Test(n/2)+8` makes one recursive call and has $T(n/2) + \Theta(1) = \Theta(\log n + 1)$ time. Two separately evaluated calls yield $2T(n/2) + \Theta(1) = \Theta(n)$. No compiler common-subexpression optimization is assumed.

Multiplication by 2 does not create a second recursive call

2 × Test(n/2) + 8

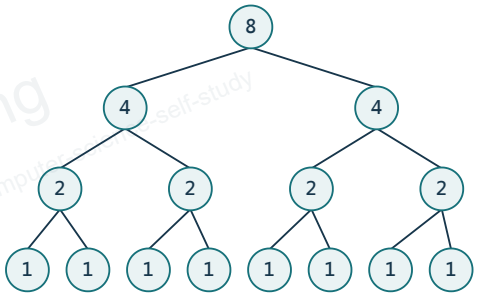
one recursive evaluation per call



4 calls including base; time Θ(log n)

Test(n/2) + Test(n/2) + 8

two recursive evaluations per call



15 calls including base; time Θ(n)

Model: source-level call count without sharing repeated calls; start n=8, base n ≤ 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Count the final failed test

For fixed $n \geq 1$ and $i = 1, \ldots, n$, start $x = n$ and repeatedly set $x \leftarrow x - i$ while $x \geq 0$. The body runs exactly $\lfloor n/i \rfloor + 1$ times, plus one failed test per i. At $n = 6$, body counts are 7, 4, 3, 2, 2, 2: total 20, or 26 condition tests.

From exact counts to growth

The body count is $S(n) = \sum_{i=1}^{n} (\lfloor n/i \rfloor + 1)$. Since $n/i - 1 < \lfloor n/i \rfloor \leq n/i$, $nH_n < S(n) \leq nH_n + n$. Integral bounds give $\log(n+1) \leq H_n \leq 1 + \log n$, hence $S(n) = \Theta(n \log n)$ for $n \geq 2$; handle the empty range separately.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Big O, lower bounds, and cost functions

$f \in O(g)$ means there exist constants $c > 0, n_0$ such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$. Ω gives the corresponding lower bound and Θ requires both. Worst-case cost is $W(n) = \max_{|x|=n} T(x)$; the symbol O alone does not mean worst case.

Worst case, expectation, amortization

Worst-case cost maximizes over legal inputs. Expected cost needs a specified input or algorithm-randomness distribution. Amortized cost bounds a sequence of operations and allocates its total among them. These are distinct statements and may coexist for one algorithm.

Duplicate keys change quicksort

With a two-way Lomuto partition that puts equal keys on one side, an all-equal input yields $n - 1$ and 0 every time; random pivot selection still costs $\Theta(n^2)$. The standard expected $\Theta(n \log n)$ claim here assumes distinct keys and random pivots. Three-way partition separates less, equal, and greater keys and does not recurse on the equal region.

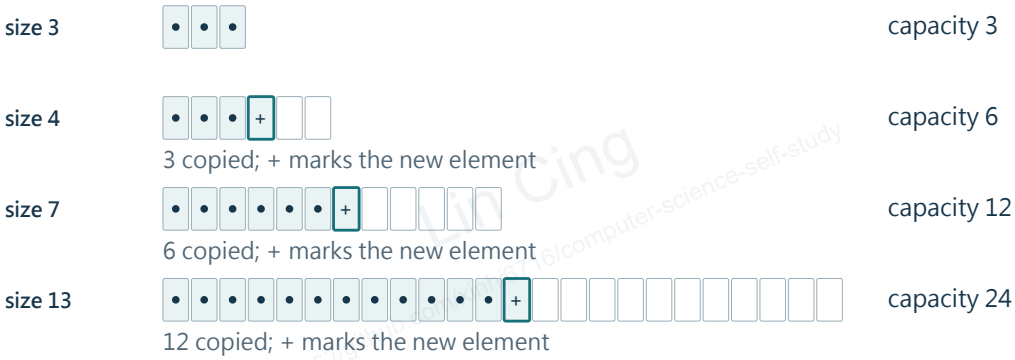
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Actual dynamic-array copies

Start with capacity $c_0 = 3$, doubling when full. Appending 13 elements copies 3, 6, and 12 elements at appends 4, 7, and 13, totaling 21. Substituting $1 + 2 + 4 + 8 = 15$ for arbitrary starting capacity undercounts this example.

Initial capacity 3; insertions 4, 7 and 13 trigger doubling

● existing + new blank = unused



Total moves = 3 + 6 + 12 = 21; amortized moves per insertion = 21/13.
Moves only; construction and allocation are separate.

Why a sequence of appends is linear

Let initial capacity $c_0 \geq 1$. At $n = 0$, both copies and writes are zero; assume $n \geq 1$ below. With no resize, the copy count is zero. Otherwise copied capacities are $c_0, 2c_0, \ldots, 2^k c_0$ and the last old capacity $c = 2^k c_0 < n$. Their geometric sum $2c - c_0 < 2n$. Including n new-element writes gives fewer than $3n$ writes. Initial allocation is excluded and must be charged separately under the allocator and construction model if counted.

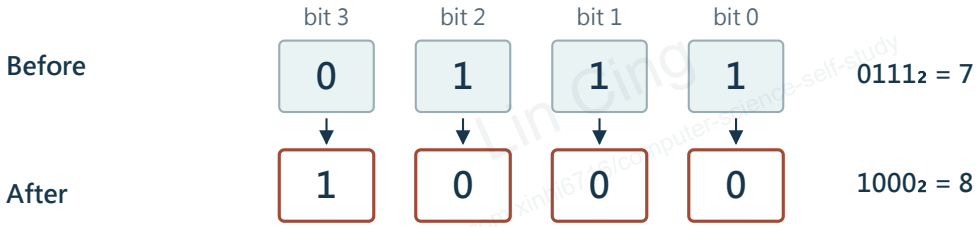
Potential method: store credit

For append-only doubling use $\Phi(s, C) = \max(0, 2s - C)$, initially zero. A non-resizing append increases potential by at most 2, giving amortized cost at most 3. At a full capacity $C > 0$, resizing costs $C + 1$ and changes potential from C to 2, giving amortized cost 3. Nonnegative potential bounds every prefix' s total actual cost by its total amortized cost.

A bit-counter potential example

Use a fixed four-bit counter modulo 16, charge one write per flipped bit, and let potential be the number of ones. $0111 \to 1000$ writes four bits and changes potential from 3 to 1, for amortized cost $4 - 2 = 2$. In general, flipping t trailing ones and setting one zero changes potential by $1 - t$, again giving 2. Wrapping all ones to zeros writes four bits and decreases potential by four.

Incrementing a binary counter: actual and amortized cost



4 bit flips: actual cost c = 4; potential Φ = number of 1 bits.

Φ before = 3, Φ after = 1; amortized cost ĉ = 4 + (1 − 3) = 2.

Assume sufficient bit width; memory allocation is excluded.

Four questions before finishing

Are empty input, duplicates and failure specified? Is there a verifiable representation invariant and abstraction function? Does the correctness argument cover preservation and termination? Does the cost claim specify the operation, input size, and assumptions for expectation or amortization? Next, apply these tools to linear structures.

References and attribution

Lin Cing, handwritten notes on data structures.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study